AI-Assisted Development Evidence

Math Practice Web Application | Selected real prompts, debugging exchanges and test results | October 2026

This evidence document uses excerpts from the student–ChatGPT project conversation and results supplied by the student. Quotes marked as excerpts are shortened; descriptions of responses are summaries, not invented transcripts. The complete chat and Git history provide additional context.

1. Initial implementation request | Phase 3

Student prompt excerpt:

> "Implement Phase 3 of the Math Practice application using the provided specification and TODO list. Add operation selection, difficulty levels, and session-length options. Preserve existing functionality, write appropriate tests, and update the TODO list."

AI response and evaluation: ChatGPT supplied a React project update with configuration controls, tests and revised TODO/development log. The student ran local checks and reported a missing @testing-library/user-event dependency, which prevented tests and build. Installing the package resolved the issue; the student reported subsequent checks passed.

2. Detailed implementation request | Revised Phase 6

Student prompt excerpt:

> "Implement the revised Phase 6: Interactive Practice System ... Preserve all existing functionality ... Prevent duplicate result recording ... Do not implement persistent data storage, the complete statistics dashboard, or mistake-review algorithms."

AI response and evaluation: ChatGPT implemented question cards, session tracking and summaries. Student terminal output showed **49/50 tests passed**. The failing test rendered multiple I Got It Wrong buttons due to insufficient test cleanup. A cleanup hook resolved that test isolation issue, while a separate unused callback parameter was removed to fix linting.

3. Debugging with concrete error evidence | Phase 8

Student-supplied evidence: The test run showed **82/84 tests passed**, while ESLint and TypeScript flagged the expression below:

```
saveDifficulty, loadPractice(difficulty);
```

Correction: ChatGPT identified the malformed expression and recommended saveDifficulty(difficulty);. The other failures involved a multiple-alert query and an outdated expectation that mistake review was unsupported. The student also independently verified that recorded mistakes could be practiced again.

4. Requirement interpretation | Self-reported correctness

Student observation: After selecting 20 for 5 + 5 and clicking I Got It Right, the correct counter increased. The student asked ChatGPT to check the actual assignment requirements. The software specification states that the learner decides whether the attempt is recorded as right or wrong. The behavior was preserved to comply with the assigned self-reported scoring design.

5. Regression testing and refinement | Phase 9

Student-reported test output: 85/87 tests passed and both linting and the production build succeeded. One test still queried a selected answer by its old accessible name, although the UI now displayed 5 ✓ Selected. Another direct-route test unexpectedly restored a saved session. Test assertions and storage setup were corrected without removing coverage. The student later reported no errors.

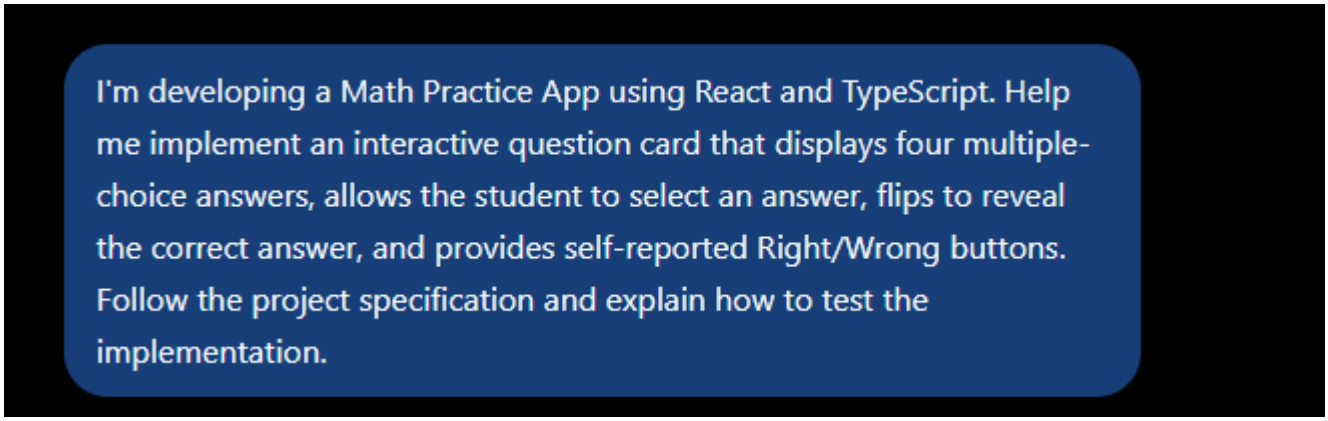
6. Final verification, audit and release

Student PowerShell results: After reinstalling dependencies in the final project, the student provided output showing **17 test files passed, 87/87 tests passed**, followed by successful ESLint and Vite production build. The subsequent Prettier check warned about 47 files. An npm audit identified three findings affecting the Vitest testing toolchain. These were disclosed rather than automatically applying a potentially breaking forced upgrade.

Manual and deployment checks: The student verified statistics reset, mistake review and mastery behavior, pushed the project to an existing class GitHub repository, deployed the project on Netlify, and reported that the deployed application worked. The assistant did not independently verify a public URL.

Optional supporting screenshots

For stronger direct evidence, append genuine screenshots of: a scoped Phase 6 prompt, the Phase 8 TypeScript error and correction, the final **87/87** terminal result, and the working Netlify application. Do not fabricate or recreate screenshots of conversations.



We'll implement a `QuestionCard` component using React, TypeScript, and `useState`.

1. Implementation

The component needs three state variables:

```
const [selectedAnswer, setSelectedAnswer] = useState<number | null>(null);
const [isFlipped, setIsFlipped] = useState(false);
const [submitted, setSubmitted] = useState(false);
```

Card behavior:

1. Display the question and four answer buttons.

2. Highlight the selected answer using `aria-pressed`.
3. Disable Flip Card until an answer is selected.
4. After flipping, display the correct answer, selected answer, and explanation.
5. Provide I Got It Right and I Got It Wrong buttons.
6. Prevent duplicate submissions.

2. Self-reported scoring

According to the specification, the student determines whether their answer counts as correct.

```
function recordResult(correct: boolean) {
  if (submitted || selectedAnswer === null || !isFlipped) return;

  setSubmitted(true);
  onResult(correct, selectedAnswer);
}
```

The parent component handles saving the result and advancing to the next question.

3. Testing

Create `QuestionCard.test.tsx` using Vitest and React Testing Library.

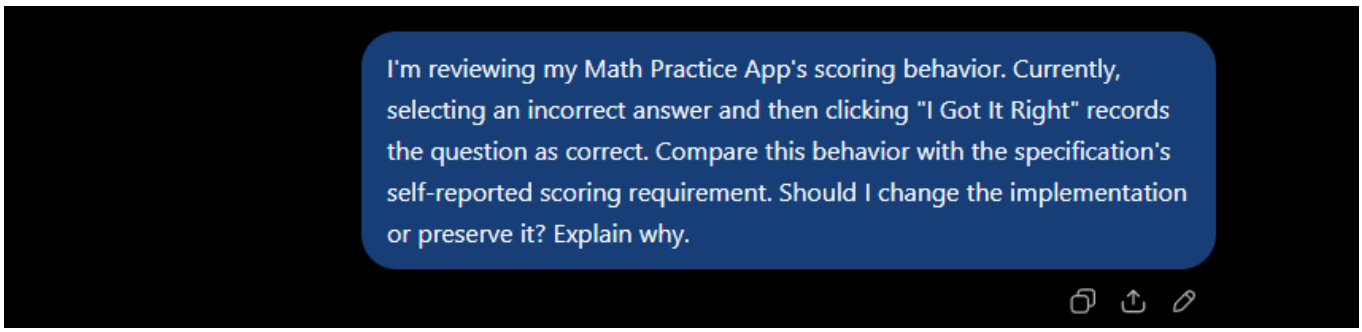
Test these scenarios:

- Four choices appear.
- Flip Card is disabled before selecting an answer.
- Selecting an answer enables flipping.
- Flipping reveals the correct answer.
- Right/Wrong buttons report the appropriate result.
- Duplicate submissions are prevented.
- A new question resets the card state.

Run:

```
npm run test
npm run lint
npm run build
```

Next step: Integrate the component into `PracticeSessionPage` while preserving the existing question-generation logic.



Recommendation: Preserve the existing implementation.

According to Section 6.3 of `specification.md`, the application uses self-reported scoring. The student decides whether an answer counts as correct or incorrect.

Why this behavior is correct

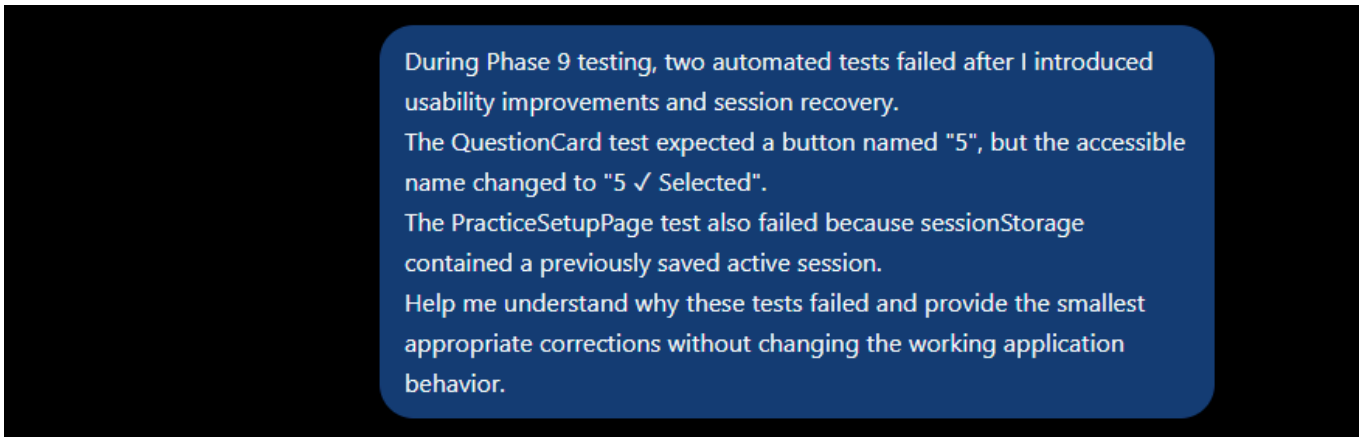
- Selecting an answer records the student's choice.
- Clicking I Got It Right records a correct attempt.
- Clicking I Got It Wrong records an incorrect attempt.
- The application does not automatically determine the recorded result by comparing answers.

Testing example

1. Select `20` for `5 + 5`.
2. Flip the card to reveal the correct answer (`10`).
3. Click I Got It Right.
4. Verify that the session's correct-answer count increases.

Although this seems counterintuitive, it follows the specification.

Decision: No code changes are necessary. Document this behavior in `DEVELOPMENT_LOG.md` as a requirements-verification decision.



Both failures were caused by changes to expected behavior and test state, not necessarily bugs in the application.

1. QuestionCard test failure

Cause: The button's accessible name changed from `5` to `5 ✓ Selected` after adding visual selection feedback.

Fix: Update the test to recognize the new label.

screen.getByRole('button', { name: /5.*selected/i });

This preserves the improved accessibility behavior.

2. PracticeSetupPage test failure

Cause: Phase 9 introduced session recovery using `sessionStorage`. A saved session interfered with a test expecting no active

session.

Fix: Clear browser storage at the beginning of that test.

window.localStorage.clear();

window.sessionStorage.clear();

This ensures the test starts with a clean state.

3. Verification

After making these corrections, run:

npm run test

npm run lint

npm run build

Conclusion: These are test maintenance fixes. No application behavior needs to change. Record the original failures, corrections, and actual rerun results in `DEVELOPMENT_L`

Implement the revised **Phase 9: Comprehensive Testing & UI Polish** of the Math Practice application using `specification.md` and `TODO.md`. Preserve all existing functionality from Phases 1–8.

1. Comprehensive Testing

- Review the entire application against the software specification and original TODO list.
- Test all four arithmetic operations across all difficulty levels.
- Verify question generation, answer selection, card flipping, and self-reported scoring.
- Test fixed-length and continuous practice sessions.
- Verify session summaries, statistics, mistake tracking, review prioritization, and mastery progression.
- Test settings persistence, browser refreshes, and data resets.
- Add automated tests for missing edge cases and integration scenarios.
- Fix any actual bugs discovered.

2. UI and Responsive Design

...

Show more

Worked for 3m 33s >

Phase 9: Testing & UI Polish

Download Phase 9 Project (ZIP) ↗

What's improved

- Fixed the Phase 8 build error and outdated test expectation.
- Added keyboard shortcuts (1–4) for answer selection.
- Improved mobile layouts and selected-answer feedback.
- Added recovery for unfinished sessions after refreshing the page.
- Improved reduced-motion accessibility.
- Added automated tests for keyboard interaction and session recovery.
- Updated `TODO.md` and `DEVELOPMENT_LOG.md` with our Phase 8 debugging evidence.

Implement the revised **Phase 8: Mistake Review & Settings** of the Math Practice application using the provided `specification.md` and `TODO.md`.
This phase combines original Phases 11–15. Preserve all existing functionality from Phases 1–7.

1. Mistake Tracking

- Record questions that students mark as incorrect using the existing self-reported scoring system.
- Store the operation, operands, correct answer, difficulty, timestamp, incorrect count, and similar-question success count.
- Group repeated mistakes into patterns rather than creating unnecessary duplicates.
- Persist mistake patterns using the existing localStorage service.
- Ensure mistake data remains available after refreshing or reopening the application.

2. Similar Question Generation

- Generate questions similar to previously missed problems.

...

Show more

Worked for 6m 11s >

Phase 8: Mistake Review & Settings

Download Phase 8 Project (ZIP) ↗

What's implemented

- **Mistake tracking:** Records incorrect answers and groups repeated mistakes.
- **Similar questions:** Generates related arithmetic problems for additional practice.
- **Mistake review:** Prioritizes frequently missed questions and tracks mastery.
- **Settings page:** Sound, animations, explanations, default difficulty, operations, session length, and mastery threshold.
- **Statistics:** Active mistakes, mastered mistakes, frequently missed facts, and review activity.
- **Data persistence:** Saves mistakes and settings while preserving existing practice history.
- **Testing:** Added automated tests for mistake generation, mastery, storage, and settings.
- **Documentation:** Updated `TODO.md` and `DEVELOPMENT_LOG.md`, including our Phase 7 self-reported scoring discussion.

Complete **Phase 10: Deployment & Final Submission** of the Math Practice application using `specification.md`, `TODO.md`, `DEVELOPMENT_LOG.md`, and the assignment instructions.
Preserve all functionality from Phases 1–9.

1. Final Project Verification

- Review the completed application against the assignment requirements.
- Confirm all required features are implemented.
- Verify the project builds successfully and automated tests pass.
- Check for unnecessary files, missing dependencies, and configuration issues.
- Do not introduce unnecessary feature changes.

2. GitHub Repository

- Review the `.gitignore` file to ensure dependencies, build outputs, and sensitive files are excluded.
- Finalize the README with project information, installation instructions, testing commands, and build instructions.

...

Show more

Worked for 3m 44s

Phase 10: Final Submission Preparation

I've prepared the final project package, deployment configuration, development report, and AI conversation evidence.

Your deliverables

1. Final Application

Complete source code, Netlify configuration, updated README, TODO, and development log

Download Final Project ZIP